COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	JULY 16 2026	**DATE SUBMITTED:**	JULY 21 2026
NAME:	DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 2.1 **Hands-on Activity 2.1 Arrays, Pointers, and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

ILO A AND B

Screenshot

Code:

```
1
2   #include <iostream>
3   #include <string.h>
4
5   class Student{
6       private:
7           std::string studentName;
8           int studentAge;
9       public:
10          //constructor
11          Student(std::string newName = "John Doe", int newAge=18){
12              studentName = (newName);
13              studentAge = newAge;
14              std::cout << "Constructor Called." <<std::endl;
15          };
16          // dconstructor
17          ~Student(){
18              std::cout << "Deconstructor called." << std::endl;
19          }
20
21          //Copy Constructor
22          Student (const Student &copyStudent){
23              std::cout << "Copy Constructor Called" << std::endl;
24              studentName = copyStudent.studentName;
25              studentAge = copyStudent.studentAge;
26          }
27
28          //Display Attributes
29          void printDetails(){
30              std::cout << this->studentName<<" "<<this->studentAge <<std::endl;
31          }
32
33  };
34
35  int main(){
36
37      Student student1("Dustin", 18);
38      Student student2(student1);
39      Student student3;
40      student3 = student2;
41
42      return 0;
43  }
```

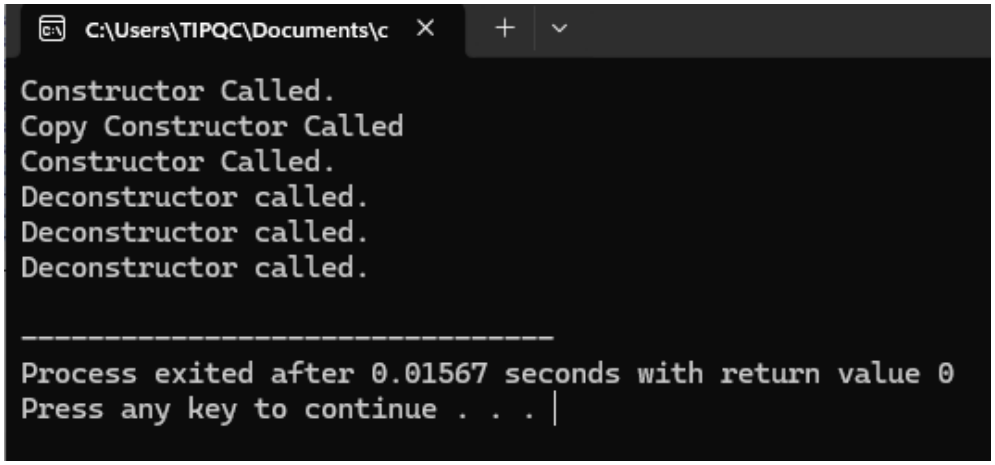
	Output: 
Observation	When running the code, first for student1, the constructor will be called and store the name and the age. Second, for student2 the copy constructor will be called since it is using the value of an existing object. And then for student3, the constructor will be called and since there was no argument passed it uses the default parameters. Next, is that it will copy the values of student2 to student3. After that, student3 will deconstruct, followed by student2, and then finally student1 will deconstruct.

Table 2.1 Initial Driver Program

Screenshot	Code:

```cpp

#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge=18){
            studentName = (newName);
            studentAge = newAge;
            std::cout << "Constructor Called." <<std::endl;
        };
        // dconstructor
        ~Student(){
            std::cout << "Deconstructor called." << std::endl;
        }

        //Copy Constructor
        Student (const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails(){
            std::cout << this->studentName<<" "<<this->studentAge <<std::endl;
        }

};

int main(){

    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

Output:

```
C:\Users\TIPQC\Documents\c

Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Deconstructor called.
Deconstructor called.
Deconstructor called.
Deconstructor called.
Deconstructor called.

--------------------------------
Process exited after 0.01018 seconds with return value 0
Press any key to continue . . .
```

Observation	When running the code, the program defined an array with a size of five. Next, is that it declares an array of Students objects that is empty with the size of five thus calling the constructor five times. Next, is that after return 0, all the created variables will be gone hence calling the deconstructor five times.

Table 2.2 Modified Driver Program with student list

Loop A	`for(int i = 0; i < j; i++){ //loop A`
	`    Student *ptr = new Student(namesList[i], ageList[i]);`
	`    studentList[i] = *ptr;`
	`}`

Observation	In this loop, the loop iterates the values inside the array named “namesList” and “ageList” which calls the constructors fie times.

Loop B	`for(int i = 0; i < j; i++){ //loop B`
	`    studentList[i].printDetails();`
	`}`

Observation	In this loop, it only printed the details of each student.
Output	Code:

```cpp

#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge=18){
            studentName = (newName);
            studentAge = newAge;
            std::cout << "Constructor Called." <<std::endl;
        };
        // deconstructor
        ~Student(){
            std::cout << "Deconstructor called." << std::endl;
        }

        //Copy Constructor
        Student (const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails(){
            std::cout << this->studentName<<" "<<this->studentAge <<std::endl;
        }

};

int main(){

    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++){ //loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
}
    for(int i = 0; i < j; i++){ //loop B
        studentList[i].printDetails();
}

    return 0;
}
```

Output:

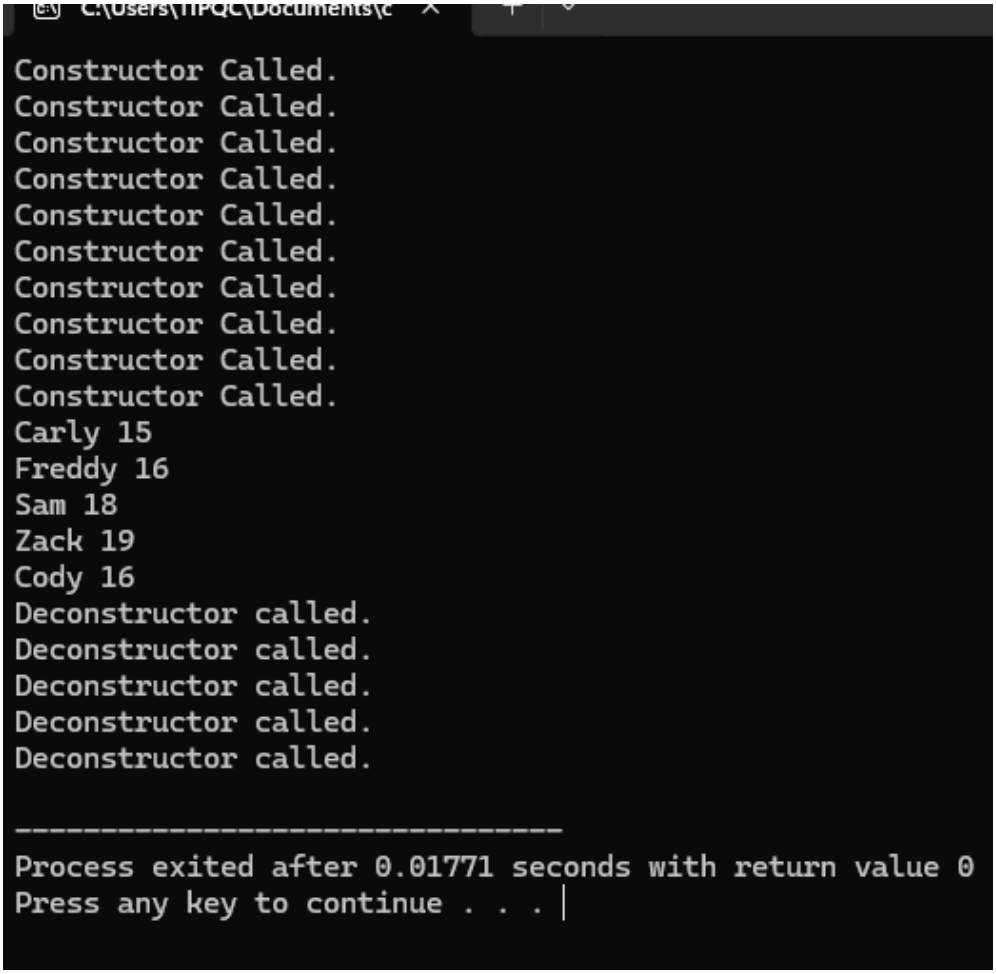
	
Observation	When running the code, first the code initialized a constant size of five for “j” and then, initialized an array with the size of five using “j”. This created an allocated memory for the 5 students thus calling the constructors five times. After that, the array “nameslist” and “ageList” is declared and then loop A is done by iterating the values inside the array named “namesList” and “ageList” which calls the constructors five times since it is being stored on the array “studentList”. Next is loop B which displays the student details. After that the five student allocated objects will deconstruct hence calling the deconstructor five times.

Table 2.3 Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C

Code:

```cpp
#include <iostream>
#include <string>

class Grocery {
    protected:
        std::string name;
        int price;
        int quantity;
    public:
        //Constructor
        Grocery(std::string theName, int thePrice, int theQuantity) {
            name = theName;
            price = thePrice;
            quantity = theQuantity;

        }
        void display() {
            std::cout << name << " " << quantity << "x " << "Php " << price << std::endl;
        }

        // copy constructor
        Grocery(const Grocery& other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
```

```
27
28          // copy assignment operator
29          Grocery& operator=(const Grocery& other) {
30              if (this != &other) {
31                  name = other.name;
32                  price = other.price;
33                  quantity = other.quantity;
34              }
35              return *this;
36          }
37
38          int calcSum() {
39              int sum = price * quantity;
40              return sum;
41          }
42
43          // deconstructor
44      virtual ~Grocery() {
45              std::cout <<  name << " has been DESTRUCTED" << std::endl;
46          }
47
48
49
50  };
51
```

```cpp
class Fruit:public Grocery {
    public:
    Fruit(std::string theName, int thePrice, int theQuantity): Grocery( theName, thePrice, theQuantity) {}

};

class Vegetable:public Grocery {
    public:
    Vegetable(std::string theName, int thePrice, int theQuantity): Grocery( theName, thePrice, theQuantity) {}


};

int totalSum(Grocery* List[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        if (List[i] != nullptr) {
            sum += List[i]->calcSum();
        }
    }
    return sum;

}
```

```cpp
int main () {
    Grocery* GroceryList[4];
    GroceryList[0] = new Fruit( theName: "Apple", thePrice: 10, theQuantity: 7);
    GroceryList[1] = new Fruit( theName: "Banana", thePrice: 10, theQuantity: 8);
    GroceryList[2] = new Vegetable( theName: "Broccoli", thePrice: 60, theQuantity: 12);
    GroceryList[3] = new Vegetable( theName: "Lettuce", thePrice: 50, theQuantity: 10);

    std::cout << "Jenna's Grocery List:\n "<< std::endl;
    for (int i = 0; i < 4; i++) {
        GroceryList[i]->display();
        std::cout<<"Total: " <<GroceryList[i]->calcSum()<<std::endl;
    }

    std::cout<<"\nTotal Bill: "<< "Php "<<totalSum(GroceryList, size: 4)<<std::endl;

    delete GroceryList[3];
    GroceryList[3] = nullptr;
    std::cout<<" "<<std::endl;

```

```
    // Loop without lettuce
    for (int i = 0; i < 4; i++) {
        if (GroceryList[i] != nullptr) {
            GroceryList[i]->display();
            std::cout<<"Total: " <<GroceryList[i]->calcSum()<<std::endl;
        }
    }

    std::cout<<"\nTotal Bill: "<< "Php "<<totalSum(GroceryList, size: 4)<<std::endl;

return 0;
}
```

Output:

```
"C:\Users\Dread\Desktop\cpe 010\HOA2
Jenna's Grocery List:

Apple 7x Php 10
Total: 70
Banana 8x Php 10
Total: 80
Broccoli 12x Php 60
Total: 720
Lettuce 10x Php 50
Total: 500

Total Bill: Php 1370
Lettuce has been DESTRUCTED

Apple 7x Php 10
Total: 70
Banana 8x Php 10
Total: 80
Broccoli 12x Php 60
Total: 720

Total Bill: Php 870

Process finished with exit code 0
```

Code Analysis:
The class Grocery creates the base class for every product on Jenna's list, it has the variable name, price, and quantity and it is all protected so that only the child classes like Fruit and Vegetable can only access it. Furthermore, the public section has the constructor which takes the name, the price, and the quantity of the item. Next is the display function which will be used to display the item's details later on the driver program. Next is the copy constructor which makes a new object by copying the variables of an existing object without modifying the original object. Next is the copy assignment operator which copies the data between objects that already exist. In connection with this, it will check if both objects are the same, if it is it will skip the copying steps and just return the object. Moving on, the function calcSum() returns the subtotal of an item, and then the deconstructor which uses the virtual keyword to properly delete an object and prevent memory leaks by deleting the child layers first. Next is the class Fruit and Vegetable which inherits all of Grocery variables and public operations. Moving on to the driver program, I first allocated an array with four empty slots that is designed to store Grocery objects. Next is that I dynamically created Fruit and Vegetable objects storing their details in a dynamic array. After that I began displaying Jenna's grocery list by using a for loop which displays all the items inside the list and their corresponding details and calculates their subtotal with the use of the arrow operator and the function calcSum(). After the loop the total bill of the items are displayed. After that, I deleted lettuce and its details which are stored on index 3, this de-allocates the heap memory that is assigned to lettuce. And then I assigned nullptr to GroceryList[[3] so that it will be empty and not active anymore. I then used a for loop to display all the items again, which does not include the lettuce anymore. After the loop, the total bill is displayed again.

Conclusion:
In conclusion I learned about implementing static and dynamic memory allocation. Moreover, I also created dynamically allocated objects using pointers and arrays. In connection to this, I also solved programming problems using dynamic memory allocation, arrays, and even pointers. Furthermore, I also did code analysis for the procedure in the observation, and also in the supplementary activity. With that, I think I did well in this activity but I have more to learn and I strive to be a better programmer.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Explanation, and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used gemini as a tool for understanding, and guide on some parts**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.